Preemptics

TECHNICAL ASSESSMENT

AI Agent OS-Level Monitoring
with eBPF & AgentSight

Technical challenge — Backend / Systems Engineering

Objective: build a minimal OS-level security sensor for an AI agent, using AgentSight and eBPF, capable of observing system activity without modifying the agent application's source code.

1. Context

AI agents can invoke LLMs and then perform actions on a Linux host: launch processes, access files, make network connections, or modify data. Application logs alone do not necessarily provide an independent view of what was actually executed by the operating system.

The goal of this exercise is to explore how OS-level observability can be used to reconstruct an agent's activity and identify potentially sensitive actions.

2. Reference technology: AgentSight

Use AgentSight as the starting point for the exercise. Study its architecture, source code, capture mechanisms and existing event model before implementing your extension.

Official AgentSight repository: https://github.com/eunomia-bpf/agentsight

AgentSight documentation: https://eunomia.dev/agentsight/

You may use the AgentSight documentation, repository documentation and relevant upstream eBPF documentation as technical references. Clearly identify what you reuse, modify or implement yourself.

3. Challenge

Develop a minimal extension around AgentSight that detects and reports a sensitive action performed by an AI agent at the Linux operating-system level.

Choose at least one primary event type:

- Process execution (execve or an equivalent process-execution observation).
- Sensitive file access.
- Network connection.
- File deletion or modification.

You are encouraged to combine process execution and file access if this can be implemented cleanly within the available time.

4. Part A — Understand the AgentSight architecture

Before coding, produce a short architecture description explaining the path from the Linux kernel to the user-space collector and the final representation of an event.

```
Linux Kernel
   ↓
eBPF Probe
   ↓
Event / Ring Buffer
   ↓
User-space Collector
   ↓
Event Model
   ↓
Correlation
   ↓
Storage / JSON
   ↓
AgentSight / Report
```

Your submission must identify, in the AgentSight codebase, the relevant components for:

- eBPF programs / probes
- event collection
- user-space processing
- agent/session identification
- persistence or export
- visualization or reporting

5. Part B — Implement an eBPF system-event probe

Implement a probe that captures process execution events and sends them from kernel space to user space. The event should contain enough information to identify the process and the action.

At minimum, expose information equivalent to:

```
{
  "timestamp": "...",
  "pid": 1234,
  "ppid": 1200,
  "uid": 1000,
  "comm": "python",
  "executable": "/usr/bin/python3",
  "command": "python agent.py"
}
```

The implementation should also demonstrate how child processes can be associated with the original agent process, for example:

```
python agent.py
  │
  ├── bash
  │    └── curl
  ├── git
  └── python
```

Explain the main design choices you make, including:

- chosen eBPF hook / attachment mechanism
- kernel-to-user-space communication mechanism
- event structure
- process-tree or process-session tracking
- error handling and event loss considerations

6. Part C — Build an Agent Session model

Create a minimal representation of an AI-agent session that allows system events to be associated with the agent that initiated them.

```
Agent Session
    │
    ├── PID
    ├── start_time
    ├── command
    ├── parent PID
    ├── processes
    ├── files
    └── network events
```

Demonstrate how the following events can be represented as one session timeline:

```
10:01:02  LLM request
10:01:05  execve("/bin/bash")
10:01:05  execve("curl")
10:01:06  connect("api.example.com:443")
10:01:07  openat("/tmp/result.txt")
10:01:07  write("/tmp/result.txt")
```

A key requirement is to explain how you determine that child processes and their events belong to the same agent session.

7. Part D — Detect a sensitive action

Implement at least one simple security rule that generates a security event when an AI-agent session performs a predefined sensitive action.

Examples include:

- execution of a sensitive command such as curl, wget, ssh, sudo, chmod or rm
- access to a sensitive path such as /etc/passwd, /etc/shadow, ~/.ssh/ or a .env file

Example output:

```
{
  "type": "AI_AGENT_SECURITY_EVENT",
  "severity": "HIGH",
  "session_id": "42",
  "pid": 4312,
  "action": "FILE_ACCESS",
```

```
  "target": "/home/user/.ssh/id_rsa"
}
```

The first version does not need to block the action. Focus on reliable detection, event quality and explainability.

8. Part E — Correlate LLM activity with OS activity

Use the capabilities already provided by AgentSight as the basis for correlating an LLM interaction with subsequent operating-system activity.

```
LLM Request
    ↓
Agent decision
    ↓
Process execution
    ↓
File / network activity
```

For example, demonstrate a timeline in which an LLM interaction is followed by a process execution and a file or network operation.

```
PROMPT
"Download the report and save it locally"

      ↓

LLM CALL

      ↓

PROCESS
curl https://...

      ↓

FILE
/tmp/report.pdf
```

Perfect semantic interpretation of the prompt is not required. The important part is the design of the correlation mechanism and its limitations.

9. Part F — Expose the data through a backend API

Provide a minimal backend interface exposing the collected agent activity. The implementation can use the language and framework you are most comfortable with.

At minimum, provide endpoints equivalent to:

```
GET /agents
GET /agents/{id}
GET /agents/{id}/timeline
GET /agents/{id}/processes
```

```
GET /agents/{id}/security-events

GET /events?pid=4312
GET /events?severity=HIGH
```

The API should make it possible to inspect an agent session and understand what the operating system actually observed.

10. Performance and scalability

Assume that the system can generate a large number of kernel events. Explain how your implementation would behave under load and what you would change if the collector started losing events.

Discuss, where relevant:

- filtering in kernel space
- ring buffer / perf buffer behavior
- batching
- backpressure
- sampling or aggregation
- memory consumption
- CPU overhead
- event loss and observability of event loss

11. Expected deliverables

- Source code for your implementation.
- A README explaining how to build, run and test the solution.
- A short architecture diagram.
- A description of your eBPF design choices.
- At least one reproducible demonstration showing an AI-agent process generating a detected security event.
- A small set of automated tests where practical.
- A short section describing limitations, assumptions and possible production improvements.

12. Technical references

- **AgentSight — official repository:** https://github.com/eunomia-bpf/agentsight
- **AgentSight documentation:** https://eunomia.dev/agentsight/
- **eBPF Developer Tutorial:** https://github.com/eunomia-bpf/bpf-developer-tutorial